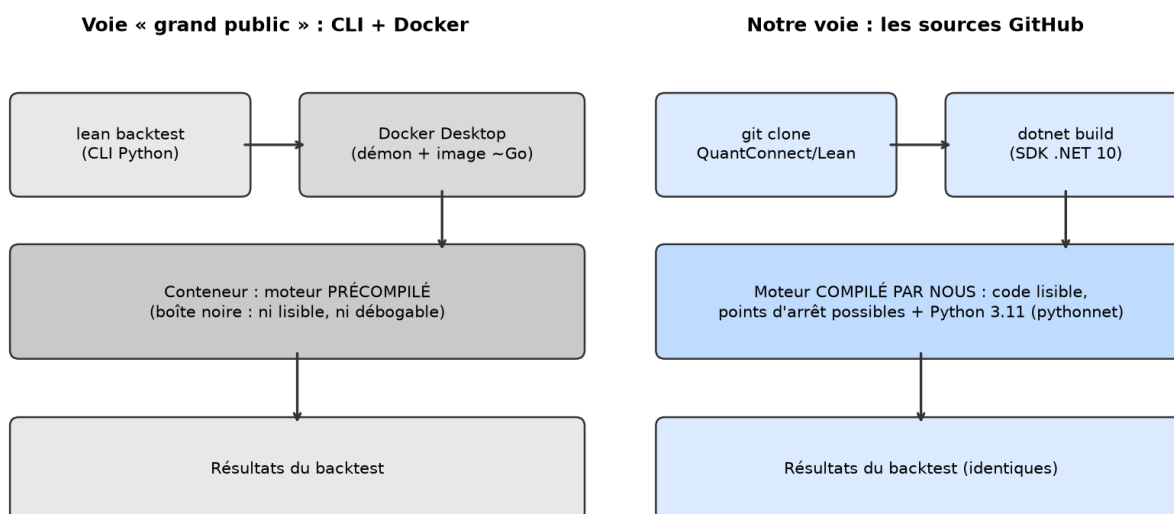


Moteur 1 — Compiler et lancer LEAN sans Docker

QuantConnect LEAN est un moteur de backtesting **événementiel** : il rejoue le marché barre par barre et fait vivre à l’algorithme exactement ce qu’il vivrait en réel — données qui arrivent, ordres qui partent, fills, frais. La voie grand public pour l’exécuter en local est la CLI `lean`, qui fait tourner un moteur précompilé dans un conteneur Docker. Nous prenons l’autre voie : **cloner les sources GitHub et compiler nous-mêmes**. Le moteur devient un simple processus .NET, lisible et débogable — quand un fill surprendra, on ouvrira le code qui l’a produit. Pour une formation dont le cœur est « comprendre pourquoi les moteurs divergent », ce n’est pas un luxe, c’est la condition.

	CLI lean + Docker	Sources GitHub compilées
Installation	<code>pip install lean</code> + Docker Desktop	<code>git clone</code> + SDK .NET
Moteur	image précompilée (boîte noire)	code source lisible et débogable
Poids	images de plusieurs Go + démon Docker	~500 Mo de sources, un simple processus
Démarrage	Docker doit tourner	instantané
Pédagogie	on apprend une CLI	on apprend un moteur



Deux chemins, le même moteur — mais un seul se laisse ouvrir, lire et déboguer.

Deux façons d’exécuter LEAN : la voie CLI+Docker (moteur précompilé, opaque) et la voie sources GitHub (moteur compilé par nous, lisible et débogable). Les deux produisent les mêmes résultats.

Figure — Deux chemins, le même moteur : seul celui de droite se laisse ouvrir, lire et déboguer. (Script : `fig_docker_vs_source.py`)

Environnement de référence : **Windows 10/11 + PowerShell**. Sous Linux/macOS, seuls les chemins et la syntaxe des variables d’environnement changent (indiqué au passage).

L’outillage

Trois outils suffisent : Git (cloner), le SDK .NET 10 (compiler — LEAN cible `net10.0`) et conda (fournir un Python 3.11 isolé pour les algos Python).

```
git --version          # attendu : git version 2.x
dotnet --list-sdks      # attendu : une ligne 10.0.xxx
conda --version         # attendu : conda 2x.x
```

S'il manque un outil (winget install Git.Git, winget install Microsoft.DotNet.SDK.10, winget install Anaconda.Miniconda3), installe puis rouvre le terminal. Vérifie que dotnet --list-sdks affiche bien un SDK **10.x** — pas seulement 8.x ou 9.x.

Côté Python : LEAN exige **Python 3.11 précisément**, car pythonnet (le pont C# ↔ Python embarqué dans le moteur) est compilé contre cette version. Un environnement conda dédié suffit — il contient sa propre python311.dll, sans toucher au Python de la machine :

```
conda create -n backtesting python=3.11 -y
conda activate backtesting
pip install pandas "wrapt==1.16.0"
python -c "import sys; print(sys.prefix)"
# ex. : C:\Users\<toi>\anaconda3\envs\backtesting <- note ce chemin, il sert plus
bas
```

pandas et wrapt (version épinglée 1.16.0) sont exigés par le pont pandas de LEAN.

Cloner et compiler le moteur

--depth 1 ne télécharge que l'état actuel du dépôt (~500 Mo tout de même : il embarque des **données d'exemple**, dont le SPY qui va servir à valider l'installation) :

```
git clone --depth 1 https://github.com/QuantConnect/Lean.git lean
cd lean
```

Le tour du propriétaire :

Dossier	Rôle
Engine/	le cœur : boucle événementielle, fills, frais
Launcher/	le point d'entrée qu'on compile et lance
Launcher/config.json	le fichier qui pilote tout (algo, langage, données)
Algorithm.Python/ · Algorithm.CSharp/	des centaines d'algos d'exemple
Data/	données d'exemple (SPY, forex, crypto...)

On ne compile que le projet **Launcher** — dotnet compile automatiquement tout ce dont il dépend (Engine, Common...). Première compilation = restauration NuGet incluse, ~1 à 2 minutes :

```
dotnet build Launcher/QuantConnect.Lean.Launcher.csproj -c Release
```

Des **milliers d'avertissements** de style défilent — c'est normal sur ce dépôt, ne les lis pas. Seule la fin compte :

```
4351 Avertissement(s)
0 Erreur(s)
```

Premier backtest natif (C#)

Avant de brancher Python, on valide la chaîne la plus courte. Par défaut, config.json pointe sur BasicTemplateFrameworkAlgorithm (C#) : un achat de SPY sur les données embarquées.

Un point qui piégera tout le monde une fois : config.json utilise des chemins **relatifs** (ex. ../../../../Data/) — on se place donc dans Launcher/bin/Release **avant** de lancer.

```
cd Launcher/bin/Release
dotnet QuantConnect.Lean.Launcher.dll
```

Le moteur déroule le backtest en quelques secondes puis imprime ses statistiques :

```
STATISTICS:: Total Fees $10.32
STATISTICS:: Portfolio Turnover 59.86%
...
Engine.Main(): Analysis Complete.
Engine.Main(): Press any key to continue.
```

Le Press any key to continue. final n'est pas un blocage : appuie sur Entrée. Pas de Docker, pas de démon — un processus .NET qui démarre et se termine.

Brancher Python 3.11

Pour un algo Python, LEAN doit savoir **quelle DLL Python charger** : c'est PYTHONNET_PYDLL. Avec un Python conda, il faut aussi PYTHONHOME — sinon la DLL se charge mais ne trouve pas sa bibliothèque standard. On pose les deux **dans le terminal, au lancement** (pas de variable système), et on surcharge les clés de config.json **en ligne de commande** — le dépôt cloné reste intact :

```
$env:PYTHONNET_PYDLL = "C:\Users\<toi>\anaconda3\envs\backtesting\python311.dll"
$env:PYTHONHOME      = "C:\Users\<toi>\anaconda3\envs\backtesting"
```

```
dotnet QuantConnect.Lean.Launcher.dll `
  --algorithm-language Python `
  --algorithm-type-name BasicTemplateAlgorithm `
  --algorithm-location "../..../Algorithm.Python/BasicTemplateAlgorithm.py"
```

Linux/macOS : export

PYTHONNET_PYDLL=\$HOME/miniconda3/envs/backtesting/lib/libpython3.11.so (.dylib sur macOS), même principe.

Sortie attendue — les statistiques canoniques de cet algo démo :

```
STATISTICS:: Net Profit 1.692%
STATISTICS:: Sharpe Ratio 8.854
STATISTICS:: Total Fees $3.44
```

Net Profit 1.692% exactement = installation conforme au comportement de référence du moteur.

C'est notre premier « chiffre de conformité » — la formation en utilisera d'autres.

Il est possible qu'une exception GIL must always be released... apparaisse **après** « Analysis Complete » : elle survient pendant le nettoyage du processus (quirk connu du pont pythonnet au shutdown). Les statistiques et les fichiers de résultats sont **intacts**.

Les fichiers de sortie

LEAN n'affiche pas que du texte : il écrit des **JSON complets** à côté du Launcher (ls BasicTemplateAlgorithm*.json). Ce sont eux qu'on exploitera dans toute la suite :

Fichier	Contenu
<Algo>.json	séries complètes (équité, benchmark...)
<Algo>-summary.json	les statistiques finales
<Algo>-order-events.json	chaque ordre, chaque fill — notre futur outil d'audit
<Algo>-log.txt	le journal complet de l'algo (la référence chronologique)

Ce qu'il faut retenir

- LEAN ne nécessite **pas** Docker : `git clone + dotnet build` suffisent (SDK .NET 10), et le moteur obtenu est **lisible et débogable**.
- Les algos Python exigent **Python 3.11 exactement** (pythonnet), branché par deux variables posées au lancement : `PYTHONNET_PYDLL + PYTHONHOME`.
- `config.json` pilote tout, ses chemins sont **relatifs** (toujours lancer depuis `Launcher/bin/Release`), et toute clé se surcharge en ligne de commande.
- Résultats = quatre fichiers (`.json`, `-summary.json`, `-order-events.json`, `-log.txt`) posés à côté du Launcher.

Suite : [Moteur 2 — Le moteur à l'œuvre et nos données](#), où l'on écrit son premier algorithme, où l'on mesure ce que le moteur fait vraiment de nos ordres, et où l'on branche nos données BTCUSDT.